

Software Design Document

I2C Driver Module

Lab 05 – Microcontrollers and Embedded Systems

Module	I2C Driver
Version	1.0.0
Author(s)	J.M. Páez Sandoval, L.A. Lugo Muñiz, D. Lopez Moreno, C.A. Martínez Macías
Institution	CETYS Universidad
Date	29/05/2026

1. Introduction

1.1 Purpose

This document describes the design and architecture of the `i2c_driver` module for Lab 05. It covers the handle-based API, initialization sequence, polling-based master transmit/receive operations, high-level register read/write helpers, error codes, and timeout handling.

1.2 Scope

The `i2c_driver` is a low-level bare-metal module for the STM32F411RE I2C1 peripheral. It operates in master mode using polling (no interrupts, no DMA). It supports both Standard Mode (100 kHz) and Fast Mode (400 kHz). All configuration is encapsulated in an `i2c_handle_t` struct. The module is used by `mpu6050.c` to read accelerometer and gyroscope data via burst reads of up to 14 bytes. GPIO configuration for SDA/SCL must be performed by the caller before `i2c_init()`. `OSPEEDR` must remain at the reset value (LOW) for correct I2C open-drain operation.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
I2C	Inter-Integrated Circuit — two-wire serial bus (SDA + SCL)
SDA	Serial Data line — open-drain, pulled high externally (4.7 kΩ on GY-521)
SCL	Serial Clock line — open-drain, pulled high externally
SM / FM	Standard Mode (100 kHz) / Fast Mode (400 kHz)
START / Sr	START condition (SDA↓ while SCL HIGH) / Repeated START
STOP	STOP condition — SDA↑ while SCL HIGH — ends transaction

Term	Definition
ACK / NACK	Acknowledge (SDA LOW on 9th clock) / Not Acknowledge
AF	Acknowledge Failure flag (SR1 bit 10) — slave sent NACK
BERR	Bus Error flag (SR1 bit 8) — misplaced START/STOP
ARLO	Arbitration Lost flag (SR1 bit 9)
BUSY	SR2 bit 1 — bus is occupied by an active transaction
CCR	Clock Control Register — determines SCL frequency
TRISE	Maximum rise time register — affects signal timing
SWRST	Software Reset — clears stuck I2C peripheral state
i2c_handle_t	Driver config struct: I2C instance, speed, APB1 clock Hz
i2c_status_t	Return type: 0=OK, negative = specific error code

1.4 Requirements

All requirements are allocated in Docs/SRS/I2C_Driver_SRS.md (provided by instructor without modification).

2. System Architecture

2.1 High-Level Design

The i2c_driver sits directly above the I2C1 hardware registers. GPIO pin configuration must be performed by the caller prior to i2c_init() to maintain separation of concerns. The driver does not configure GPIO.

Layer	Modules
Application	main.c — orchestrates sensor init and read loop
Sensor Driver	mpu6050.c — calls i2c_reg_read_burst() for 14-byte sensor reads
I2C Driver	All I2C1 register operations: START, address, data, STOP, error handling
Hardware	STM32F411RE I2C1 (APB1). PB8=SCL, PB9=SDA, AF4, open-drain, pull-up.

2.2 Module Interfaces

Public API declared in i2c_driver.h. Dependency: stm32f411xe.h only. No HAL, no gpio_driver. The caller must configure PB8/PB9 as AF4 open-drain with pull-up and leave OSPEEDR at LOW before calling i2c_init().

3. Detailed Design

3.1 Class Diagram

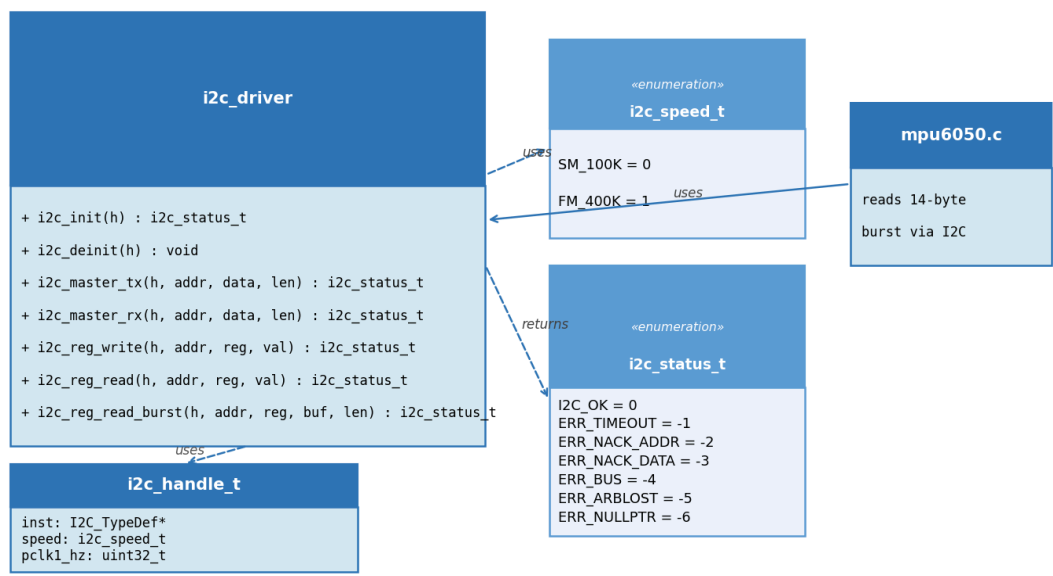


Figure 1. Class Diagram — *i2c_driver*, *i2c_handle_t*, enumerations, and *mpu6050* consumer

3.2 Module Behavior

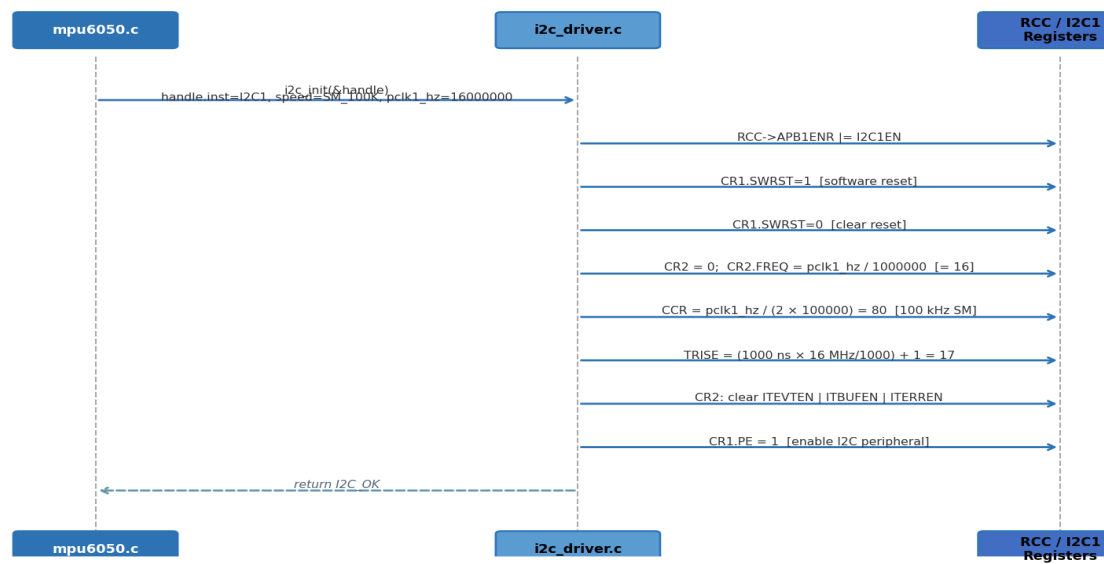


Figure 2. Sequence Diagram — *i2c_init()*: clock enable, *SWRST*, *CCR/TRISE* write, *PE*=1

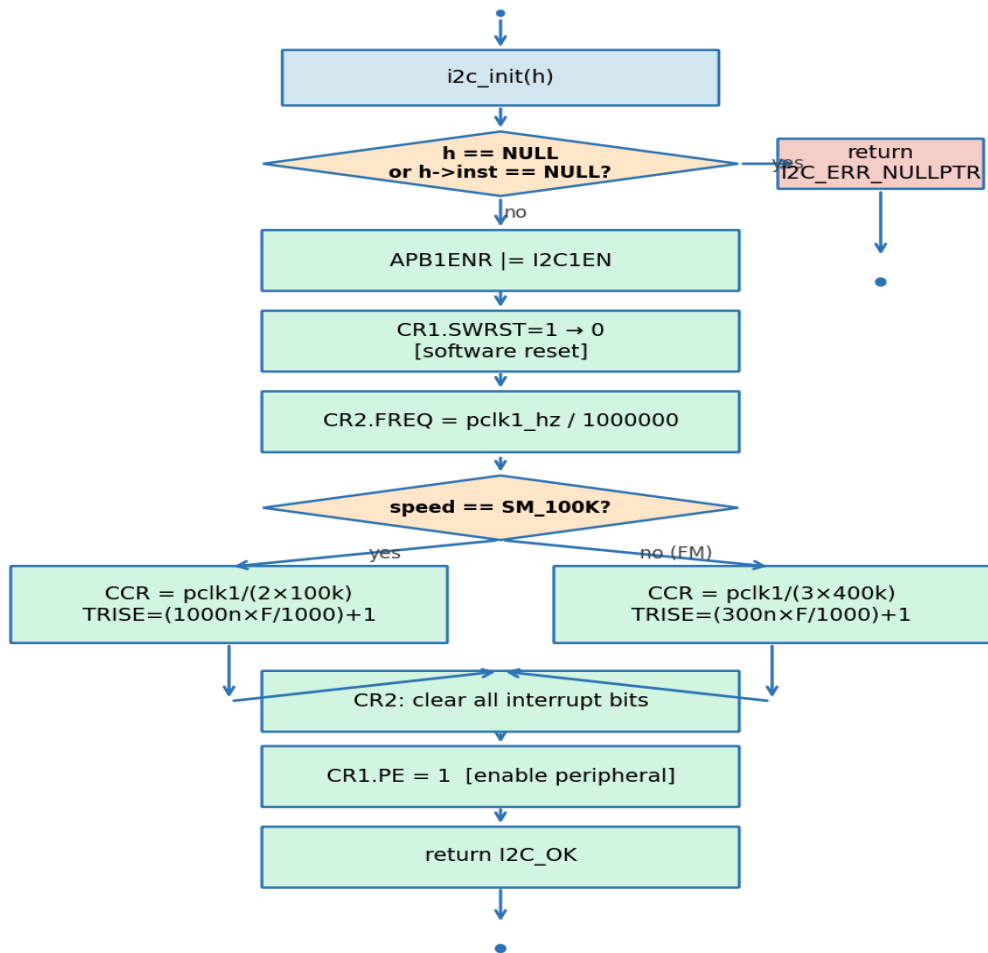


Figure 3. Activity Diagram — `i2c_init()`: NULL check, speed selection, register configuration

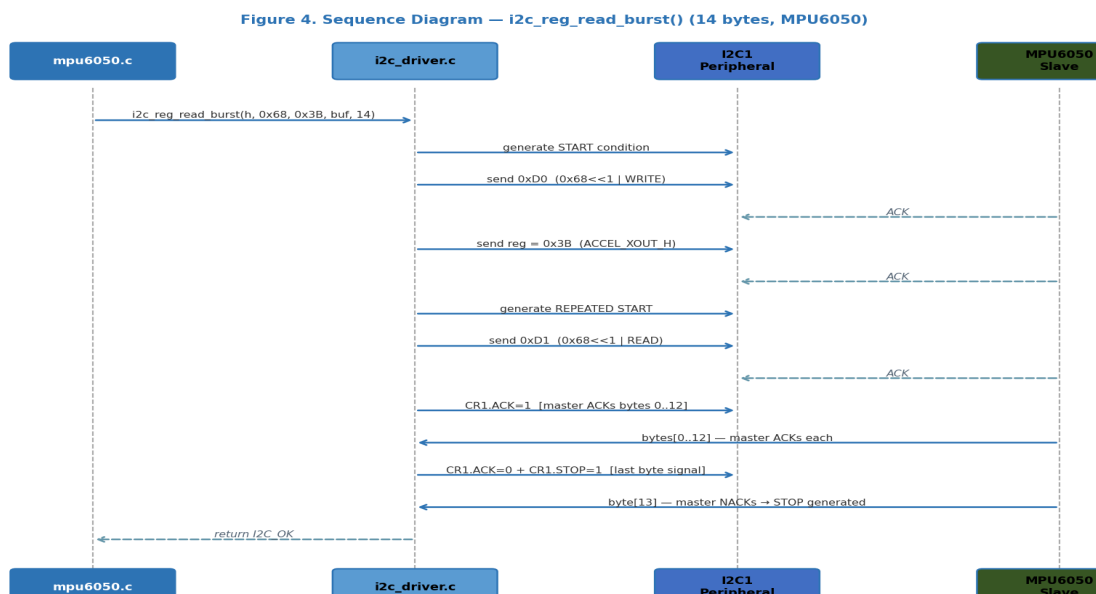


Figure 4. Sequence Diagram — `i2c_reg_read_burst()` (14 bytes, MPU6050)

Figure 4. Sequence Diagram — *i2c_reg_read_burst()*: write-then-repeated-START-read for 14 bytes

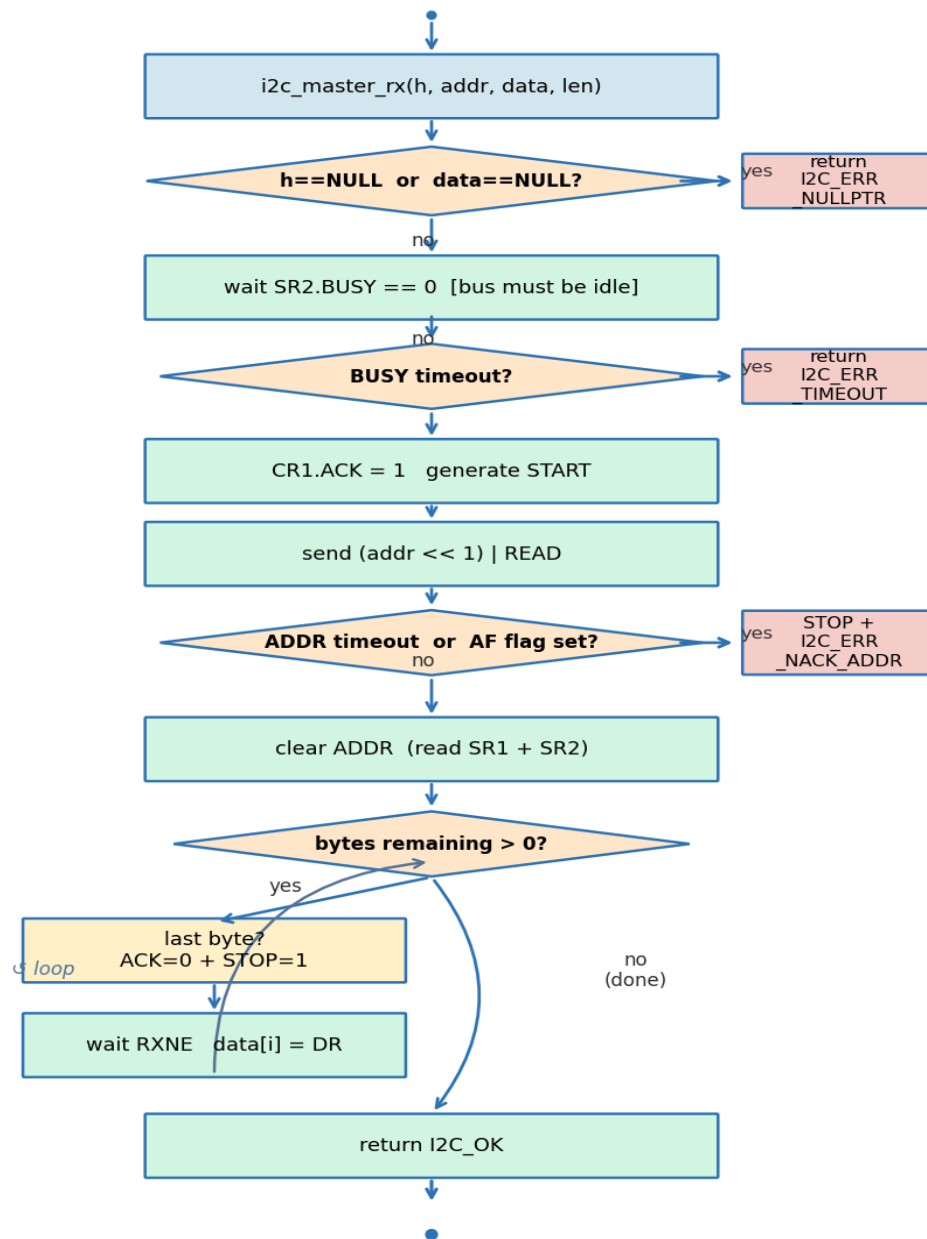


Figure 5. Activity Diagram — *i2c_master_rx()*: BUSY/ADDR timeout and NACK/STOP handling

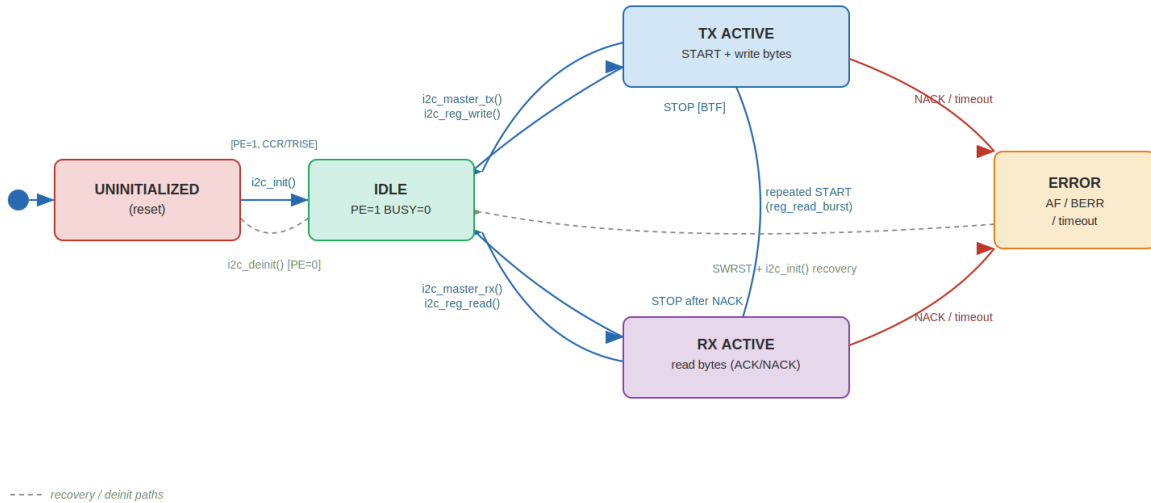


Figure 6. State Diagram — I2C Bus States

3.3 I2C Driver Module Description

`i2c_init()` validates the handle, enables APB1 clock for I2C1, performs a software reset (SWRST) to clear any stuck state, writes CR2.FREQ with the APB1 clock in MHz, computes and writes CCR and TRISE for the requested speed mode, disables all interrupts, and enables the peripheral (PE=1).

`i2c_master_tx()` generates START, sends the 7-bit slave address with W bit, transmits each byte waiting for TXE, waits for BTF, then generates STOP. All waits use a timeout counter; on expiry the function generates STOP and returns the corresponding error code.

`i2c_master_rx()` waits for BUSY=0, generates START, sends address with R bit, enables ACK, reads N-1 bytes with ACK, then disables ACK and generates STOP before reading the last byte — per the STM32 I2C master receiver protocol.

`i2c_reg_read_burst()` writes the register pointer address (START + addr+W + reg), then issues a repeated START and reads len bytes. Used by `mpu6050.c` to read all 14 sensor registers (ACCEL_XOUT_H through GYRO_ZOUT_L) in one atomic I2C transaction.

4. Application Programming Interfaces

4.1 Data Types and Macros

Type	Fields / Values	Notes
i2c_handle_t	inst: I2C_TypeDef*speed: i2c_speed_tpc1k1_hz: uint32_t	Config struct — passed by pointer to all functions
i2c_speed_t	SM_100K=0FM_400K=1	Selects CCR/TRISE formula in i2c_init()
i2c_status_t	OK=0, ERR_TIMEOUT=-1ERR_ NACK_ADDR=-2, ERR_NACK_DATA=-3ER R_BUS=-4, ERR_ARBLOST=-5ERR_ NULLPTR=-6	Negative = error; supports if(rc < 0) checks

4.2 Error Codes

Code	Value	Definition
I2C_OK	0	Operation completed successfully
I2C_ERR_TIMEOUT	-1	Polling loop reached I2C_TIMEOUT iterations without flag
I2C_ERR_NACK_ADDR	-2	Slave did not ACK its address — AF flag set
I2C_ERR_NACK_DATA	-3	Slave did not ACK a data byte during transmission
I2C_ERR_BUS	-4	Bus error detected (BERR — misplaced START or STOP)
I2C_ERR_ARBLOST	-5	Arbitration lost (ARLO flag set)
I2C_ERR_NULLPTR	-6	NULL pointer passed as handle or data buffer

4.3 Function Definitions

i2c_init

Function Name	i2c_init
Syntax	i2c_status_t i2c_init(i2c_handle_t *h)
Parameters (in)	h — pointer to caller-initialized i2c_handle_t(inst=I2C1, speed, pclk1_hz=16000000)
Return value	I2C_OK or I2C_ERR_NULLPTR
Description	Enables I2C1 APB1 clock, SWRST, configures CR2/CCR/TRISE for selected speed, clears interrupt bits, sets PE=1.
Constraints	Caller must configure PB8/PB9 as AF4 open-drain pull-up with OSPEEDR=LOW before calling.
Available via	i2c_driver.h

i2c_deinit

Function Name	i2c_deinit
Syntax	void i2c_deinit(i2c_handle_t *h)
Parameters (in)	h — handle pointer
Return value	void
Description	Clears CR1.PE=0 to disable the I2C peripheral. Does not reset GPIO.
Constraints	No NULL check. Call before speed reconfiguration.
Available via	i2c_driver.h

i2c_master_tx

Function Name	i2c_master_tx
Syntax	i2c_status_t i2c_master_tx(i2c_handle_t *h, uint8_t slave_addr, const uint8_t *data, uint16_t len)
Parameters (in)	h, slave_addr (7-bit), data — TX buffer, len — byte count
Return value	I2C_OK or error code on timeout/NACK
Description	START → addr+W → N bytes (TXE per byte) → BTF → STOP. Timeout on every wait step.
Constraints	Returns immediately with error on any timeout or NACK.
Available via	i2c_driver.h

i2c_master_rx

Function Name	i2c_master_rx
Syntax	i2c_status_t i2c_master_rx(i2c_handle_t *h, uint8_t slave_addr, uint8_t *data, uint16_t len)
Parameters (in)	h, slave_addr (7-bit), data — RX buffer, len — byte count
Return value	I2C_OK or error code
Description	Wait BUSY=0 → START → addr+R → ACK=1 → read N-1 bytes with ACK → ACK=0+STOP → last byte.
Constraints	For len=1: ACK must be disabled before clearing ADDR flag.
Available via	i2c_driver.h

i2c_reg_write

Function Name	i2c_reg_write
Syntax	i2c_status_t i2c_reg_write(i2c_handle_t *h, uint8_t slave_addr, uint8_t reg, uint8_t value)
Parameters (in)	h, slave_addr, reg — register address, value — byte to write

Return value	I2C_OK or error code
Description	Convenience wrapper: sends [addr+W, reg, value] in one transaction via i2c_master_tx.
Constraints	Single byte only. Use i2c_master_tx for multi-byte writes.
Available via	i2c_driver.h

i2c_reg_read

Function Name	i2c_reg_read
Syntax	i2c_status_t i2c_reg_read(i2c_handle_t *h, uint8_t slave_addr, uint8_t reg, uint8_t *value)
Parameters (in)	h, slave_addr, reg, value — pointer to result byte
Return value	I2C_OK or error code
Description	Writes register pointer then repeated START + reads 1 byte.
Constraints	Single register read only.
Available via	i2c_driver.h

i2c_reg_read_burst

Function Name	i2c_reg_read_burst
Syntax	i2c_status_t i2c_reg_read_burst(i2c_handle_t *h, uint8_t slave_addr, uint8_t reg, uint8_t *buf, uint16_t len)
Parameters (in)	h, slave_addr, reg — start register, buf — RX buffer, len — byte count (up to 14 for MPU6050)
Return value	I2C_OK or error code
Description	Writes register pointer, repeated START, reads len bytes in burst. Used by mpu6050.c to read ACCEL+TEMP+GYRO (14 bytes, 0x3B–0x48) in one atomic transaction.
Constraints	Slave must support register auto-increment. MPU6050 does by default.
Available via	i2c_driver.h

4.4 Internal Function Definitions

The i2c_driver has no static helper functions. All I2C protocol steps (START, ADDR, data loop, STOP, error detection) are implemented inline in the public API functions using an I2C_WAIT macro with a shared I2C_TIMEOUT constant.

5. Future Enhancements

- Add DMA-based TX/RX to free the CPU during transfers (14-byte MPU6050 burst takes ~1.26 ms at 100 kHz).
- Add interrupt-driven mode with completion callbacks for non-blocking sensor reads.
- Add 10-bit addressing support for sensors with addresses beyond 0x77.
- Implement I2C bus recovery (9-clock pulse sequence) to release a stuck SDA line after errors.